

```
In [2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import duckdb

from sklearn.preprocessing import LabelEncoder
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pickle
```

```
In [3]: sql_departments = duckdb.sql("SELECT * FROM 'departments.csv'")
sql_encounters = duckdb.sql("SELECT * FROM 'encounters.csv'")
sql_patients = duckdb.sql("SELECT * FROM 'patients.csv'")
sql_diagnosis = duckdb.sql("SELECT * FROM 'diagnosis.csv'")
sql_providers = duckdb.sql("SELECT * FROM 'providers.csv'")
sql_social_determinant = duckdb.sql("SELECT * FROM 'social_determinants.csv'")
sql_tiger = duckdb.sql("SELECT * FROM 'tigercensuscodes.csv'")
```

Exploring the Data (Departments)

```
In [4]: departments_df = sql_departments.df()
print(f"Dimensions of the Data: {departments_df.shape}")
print("-"*100)
print(departments_df.info())
```

Dimensions of the Data: (11597, 9)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11597 entries, 0 to 11596
Data columns (total 9 columns):
#   Column                Non-Null Count  Dtype
---  -
0   DepartmentKey          11597 non-null  int64
1   Address                11597 non-null  object
2   City                   11597 non-null  object
3   County                 11597 non-null  object
4   DepartmentName         11597 non-null  object
5   DepartmentSpecialty    11597 non-null  object
6   DepartmentType         11597 non-null  object
7   PostalCode             11597 non-null  object
8   CensusTract            11597 non-null  object
dtypes: int64(1), object(8)
memory usage: 815.5+ KB
None
```

NOTEABLE FEATURES

DepartmentKey: Unique key for Providers department

DepartmentSpecialty: What the Provider does

```
In [5]: departments_df["DepartmentSpecialty"].unique()
```

```
Out[5]: array(['*Deleted', '*Not Applicable', '*Unspecified',
               'Central Scheduling', 'Home Health Services', 'Hospice Services',
               'Emergency Medicine', 'Critical Care Medicine', 'Neonatology',
               'Obstetrics', 'Pediatrics', 'Hematology and Oncology',
               'Gastroenterology', 'Pain Medicine', 'Pharmacy',
               'Cardiac Rehabilitation', 'Radiology', 'Rheumatology', 'Lab',
               'Psychiatry', 'Physical Medicine and Rehabilitation',
               'Pre-Admission Testing', 'Pulmonology', 'Sleep Medicine',
               'Wound Care', 'Nutrition', 'Respiratory Therapy',
               'Case Management', 'Physical Therapy', 'Occupational Therapy',
               'Speech Therapy', 'Neurology', 'Anesthesiology', 'Otolaryngology',
               'Oncology', 'Radiation Therapy', 'Radiation Oncology',
               'Palliative Medicine', 'Infusion Therapy', 'Orthopedic Surgery',
               'Family Medicine', 'Nuclear Medicine', 'Internal Medicine',
               'Urgent Care', 'Dermatology', 'Cardiothoracic Surgery',
               'Obstetrics and Gynecology', 'Intensive Care',
               'Cardiac Intensive Care', 'General Surgery',
               'Neurological Intensive Care', 'Pediatric Intensive Care',
               'Surgical Intensive Care', 'Transplant', 'Nephrology',
               'Pediatric Gastroenterology', 'Pediatric Surgery',
               'Pediatric Emergency Medicine', 'Burn Surgery', 'Social Services',
               'Bariatrics', 'Cardiology', 'Allergy', 'Urology', 'Neurosurgery',
               'Endocrinology', 'Pediatric Endocrinology', 'Pediatric Cardiology',
               'Ophthalmology', 'Research', 'Plastic Surgery',
               'Infectious Diseases', 'Maternal and Fetal Medicine',
               'Addiction Medicine', 'Occupational Medicine',
               'Pediatric Critical Care Medicine', 'Diabetes Services',
               'Care Management', 'Pediatric Oncology',
               'Pediatric Physical Medicine and Rehabilitation',
               'Pediatric Radiology', 'Pediatric Allergy', 'Pediatric Audiology',
               'Pediatric Cardiothoracic Surgery', 'Pediatric Dermatology',
               'Developmental and Behavioral Pediatrics',
               'Pediatric Otolaryngology', 'Pediatric Genetics',
               'Pediatric Infectious Disease', 'Pediatric Nephrology',
               'Pediatric Neurology', 'Pediatric Neurosurgery',
               'Pediatric Occupational Therapy', 'Pediatric Ophthalmology',
               'Pediatric Orthopedic Surgery', 'Pediatric Physical Therapy',
               'Pediatric Plastic Surgery', 'Pediatric Psychiatry',
               'Pediatric Pulmonology', 'Pediatric Radiation Oncology',
               'Pediatric Rheumatology', 'Pediatric Social Services',
               'Pediatric Speech Therapy', 'Pediatric Urology',
               'Child and Adolescent Psychiatry', 'Genetics', 'Gerontology',
               'Gynecology', 'Pathology', 'Sports Medicine', 'Vascular Surgery',
               'Audiology', 'Dentistry', 'Chiropractic Medicine', 'Foot & Ankle',
               'Conversion', '*Unknown', 'Pediatric Hematology and Oncology',
               'NA', 'Primary Care', 'Behavioral Health'], dtype=object)
```

```
In [6]: del departments_df #To save memory going to have all the data used later through SQ
```

EXPLORING DIAGNOSIS (Diagnosis)

```
In [13]: diagnosis_df = sql_diagnosis.df()
```

```
In [14]: diagnosis_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1531262 entries, 0 to 1531261
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   DiagnosisKey           1531262 non-null int64
 1   GroupName              1531262 non-null object
 2   GroupCode              1531262 non-null object
 3   DiagnosisName          1531262 non-null object
 4   DiagnosisValue         1531262 non-null object
dtypes: int64(1), object(4)
memory usage: 58.4+ MB
```

NOTEABLE FEATURES

DiagnosisKey: Unique key for Diagnosis department

DiagnosisName: What type of diagnosis it is

```
In [15]: diagnosis_df['DiagnosisName'].unique() #grabs the first 5 unique values
```

```
Out[15]: array(['Paratyphoid fever, unspecified( ICD-10-CM: A01.4 )',
                'Primary respiratory tuberculosis( ICD-10-CM: A15.7 )',
                'Displaced fracture of pisiform, unspecified wrist, subsequent encounter fo
r fracture with nonunion( ICD-10-CM: S62.163K )',
                ...,
                'Puncture wound of abdominal wall without foreign body, left flank with pen
etration into peritoneal cavity, sequela( ICD-10-CM: S31.637S )',
                'Poisoning by fluoroquinolone antibiotics, undetermined, sequela( ICD-10-C
M: T36.AX4S )',
                'Other sharp object entering into or through a natural orifice, subsequent
encounter( ICD-10-CM: W44.H9XD )'],
                dtype=object)
```

```
In [16]: diagnosis_df['DiagnosisValue'].unique()[:5]
```

```
Out[16]: array(['A01.4', 'A15.7', 'S62.163K', 'S22.41XG', 'S62.663G'], dtype=object)
```

```
In [17]: del diagnosis_df
```

Exploring the Data (Encounters) MOST IMPORTANT

```
In [18]: duckdb.sql("DESCRIBE SELECT * FROM sql_encounters").df() #Using SQL server to descr
```

Out[18]:

	column_name	column_type	null	key	default	extra
0	Date	DATE	YES	None	None	None
1	AdmissionInstant	VARCHAR	YES	None	None	None
2	AdmitYear	VARCHAR	YES	None	None	None
3	AdmitMonth	VARCHAR	YES	None	None	None
4	AdmitDay	VARCHAR	YES	None	None	None
5	AdmitHour	VARCHAR	YES	None	None	None
6	AdmitMinute	VARCHAR	YES	None	None	None
7	AdmissionSource	VARCHAR	YES	None	None	None
8	AdmissionType	VARCHAR	YES	None	None	None
9	DischargeInstant	VARCHAR	YES	None	None	None
10	DischargeYear	VARCHAR	YES	None	None	None
11	DischargeMonth	VARCHAR	YES	None	None	None
12	DischargeDay	VARCHAR	YES	None	None	None
13	DischargeHour	VARCHAR	YES	None	None	None
14	DischargeMinute	VARCHAR	YES	None	None	None
15	EncounterKey	BIGINT	YES	None	None	None
16	PatientDurableKey	BIGINT	YES	None	None	None
17	Type	VARCHAR	YES	None	None	None
18	VisitType	VARCHAR	YES	None	None	None
19	VisitTypeDescription	VARCHAR	YES	None	None	None
20	ProviderDurableKey	BIGINT	YES	None	None	None
21	AttendingProviderDurableKey	BIGINT	YES	None	None	None
22	DischargeProviderDurableKey	BIGINT	YES	None	None	None
23	DepartmentKey	BIGINT	YES	None	None	None
24	PrimaryDiagnosisKey	BIGINT	YES	None	None	None
25	IsEdVisit	BIGINT	YES	None	None	None
26	IsHospitalAdmission	BIGINT	YES	None	None	None
27	IsHospitalOutpatientVisit	BIGINT	YES	None	None	None
28	IsInpatientAdmission	BIGINT	YES	None	None	None
29	IsObservation	BIGINT	YES	None	None	None

	column_name	column_type	null	key	default	extra
30	IsOutpatientFaceToFaceVisit	BIGINT	YES	None	None	None

```
In [19]: encounters_df = duckdb.sql("SELECT Date, AdmissionInstant, AdmitYear FROM sql_encou
encounters_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7675801 entries, 0 to 7675800
Data columns (total 3 columns):
#   Column          Dtype
---  -
0   Date            datetime64[us]
1   AdmissionInstant object
2   AdmitYear        object
dtypes: datetime64[us](1), object(2)
memory usage: 175.7+ MB
```

```
In [8]: duckdb.sql("SELECT * FROM sql_encounters WHERE PatientDurableKey = 7663504").df()
```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color
='black'))
```

```
Out[8]:   Date  AdmissionInstant  AdmitYear  AdmitMonth  AdmitDay  AdmitHour  AdmitMinute
```

0 rows × 31 columns



NOTES For Data Set

1. There are null values present in the data
2. The data is huge and may need cleaning or deeper analysis
3. Might need to merge data and narrow down the subject we are trying to look into

EXPLORING THE DATA (Patients)

```
In [22]: duckdb.sql("DESCRIBE SELECT * FROM sql_patients").df()
```

Out[22]:

	column_name	column_type	null	key	default	extra
0	CensusBlockGroupFipsCode	VARCHAR	YES	None	None	None
1	DurableKey	BIGINT	YES	None	None	None
2	FirstRace	VARCHAR	YES	None	None	None
3	MaritalStatus	VARCHAR	YES	None	None	None
4	MyChartStatus	VARCHAR	YES	None	None	None
5	OmbEthnicity	VARCHAR	YES	None	None	None
6	OmbRace	VARCHAR	YES	None	None	None
7	SexAssignedAtBirth	VARCHAR	YES	None	None	None
8	SexualOrientation	VARCHAR	YES	None	None	None
9	SmokingStatus	VARCHAR	YES	None	None	None
10	VitalStatus	VARCHAR	YES	None	None	None
11	PatientBirthYearBin	VARCHAR	YES	None	None	None

EXPLORING THE DATA (Providers)

In [23]: `duckdb.sql("DESCRIBE SELECT * FROM sql_providers").df()`

Out[23]:

	column_name	column_type	null	key	default	extra
0	DurableKey	BIGINT	YES	None	None	None
1	ClinicianTitle	VARCHAR	YES	None	None	None
2	OfficeAddress	VARCHAR	YES	None	None	None
3	OfficeCity	VARCHAR	YES	None	None	None
4	OfficePostalCode	VARCHAR	YES	None	None	None
5	PrimaryDepartment	VARCHAR	YES	None	None	None
6	PrimarySpecialty	VARCHAR	YES	None	None	None
7	Type	VARCHAR	YES	None	None	None

USING SQL TO COMBINE THE DATA (ENCOUNTERS & PATIENTS & DIAGNOSIS & PROVIDERS)

In [32]:

```
combined_data = duckdb.sql("""
    WITH duped_diagnosis AS (
        SELECT *
        FROM (
            SELECT *,
                ROW_NUMBER() OVER (PARTITION BY DiagnosisKey ORDER BY DiagnosisKey)
```

```

        FROM sql_diagnosis
    )
    WHERE rn = 1
),
duped_providers AS (
    SELECT *
    FROM (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY DurableKey, Type ORDER BY DurableKey) AS rn
        FROM sql_providers
    )
    WHERE rn = 1
),
duped_patients AS (
    SELECT *
    FROM (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY DurableKey ORDER BY DurableKey) AS rn
        FROM sql_patients
    )
    WHERE rn = 1
),
duped_departments AS (
    SELECT *
    FROM (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY DepartmentKey ORDER BY DepartmentKey) AS rn
        FROM sql_departments
    )
    WHERE rn = 1
)
SELECT
    e.*,
    p.CensusBlockGroupFipsCode, p.FirstRace, p.MaritalStatus, p.MyChartStatus,
    p.OmbEthnicity, p.OmbRace, p.SexAssignedAtBirth, p.SexualOrientation,
    p.SmokingStatus, p.VitalStatus, p.PatientBirthYearBin,
    d.GroupName, d.GroupCode, d.DiagnosisName, d.DiagnosisValue,
    pr.ClinicianTitle, pr.OfficeAddress, pr.OfficeCity, pr.OfficePostalCode,
    pr.PrimaryDepartment, pr.PrimarySpecialty,
    de.Address, de.City, de.County, de.DepartmentName,
    de.DepartmentSpecialty, de.DepartmentType, de.PostalCode, de.CensusTract
FROM sql_encounters AS e
LEFT JOIN duped_patients AS p
    ON e.PatientDurableKey = p.DurableKey
LEFT JOIN duped_diagnosis AS d
    ON e.PrimaryDiagnosisKey = d.DiagnosisKey
LEFT JOIN duped_providers AS pr
    ON e.ProviderDurableKey = pr.DurableKey AND e.Type = pr.Type
LEFT JOIN duped_departments AS de
    ON e.DepartmentKey = de.DepartmentKey
""")

duckdb.sql("SELECT COUNT(*) FROM combined_data")

```

```
FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))
```

Out[32]:

count_star() int64
7675801

In [33]: `duckdb.sql("DESCRIBE SELECT * FROM combined_data").df()`

Out[33]:

	column_name	column_type	null	key	default	extra
0	Date	DATE	YES	None	None	None
1	AdmissionInstant	VARCHAR	YES	None	None	None
2	AdmitYear	VARCHAR	YES	None	None	None
3	AdmitMonth	VARCHAR	YES	None	None	None
4	AdmitDay	VARCHAR	YES	None	None	None
5	AdmitHour	VARCHAR	YES	None	None	None
6	AdmitMinute	VARCHAR	YES	None	None	None
7	AdmissionSource	VARCHAR	YES	None	None	None
8	AdmissionType	VARCHAR	YES	None	None	None
9	DischargeInstant	VARCHAR	YES	None	None	None
10	DischargeYear	VARCHAR	YES	None	None	None
11	DischargeMonth	VARCHAR	YES	None	None	None
12	DischargeDay	VARCHAR	YES	None	None	None
13	DischargeHour	VARCHAR	YES	None	None	None
14	DischargeMinute	VARCHAR	YES	None	None	None
15	EncounterKey	BIGINT	YES	None	None	None
16	PatientDurableKey	BIGINT	YES	None	None	None
17	Type	VARCHAR	YES	None	None	None
18	VisitType	VARCHAR	YES	None	None	None
19	VisitTypeDescription	VARCHAR	YES	None	None	None
20	ProviderDurableKey	BIGINT	YES	None	None	None
21	AttendingProviderDurableKey	BIGINT	YES	None	None	None
22	DischargeProviderDurableKey	BIGINT	YES	None	None	None
23	DepartmentKey	BIGINT	YES	None	None	None
24	PrimaryDiagnosisKey	BIGINT	YES	None	None	None
25	IsEdVisit	BIGINT	YES	None	None	None
26	IsHospitalAdmission	BIGINT	YES	None	None	None
27	IsHospitalOutpatientVisit	BIGINT	YES	None	None	None
28	IsInpatientAdmission	BIGINT	YES	None	None	None
29	IsObservation	BIGINT	YES	None	None	None

	column_name	column_type	null	key	default	extra
30	IsOutpatientFaceToFaceVisit	BIGINT	YES	None	None	None
31	CensusBlockGroupFipsCode	VARCHAR	YES	None	None	None
32	FirstRace	VARCHAR	YES	None	None	None
33	MaritalStatus	VARCHAR	YES	None	None	None
34	MyChartStatus	VARCHAR	YES	None	None	None
35	OmbEthnicity	VARCHAR	YES	None	None	None
36	OmbRace	VARCHAR	YES	None	None	None
37	SexAssignedAtBirth	VARCHAR	YES	None	None	None
38	SexualOrientation	VARCHAR	YES	None	None	None
39	SmokingStatus	VARCHAR	YES	None	None	None
40	VitalStatus	VARCHAR	YES	None	None	None
41	PatientBirthYearBin	VARCHAR	YES	None	None	None
42	GroupName	VARCHAR	YES	None	None	None
43	GroupCode	VARCHAR	YES	None	None	None
44	DiagnosisName	VARCHAR	YES	None	None	None
45	DiagnosisValue	VARCHAR	YES	None	None	None
46	ClinicianTitle	VARCHAR	YES	None	None	None
47	OfficeAddress	VARCHAR	YES	None	None	None
48	OfficeCity	VARCHAR	YES	None	None	None
49	OfficePostalCode	VARCHAR	YES	None	None	None
50	PrimaryDepartment	VARCHAR	YES	None	None	None
51	PrimarySpecialty	VARCHAR	YES	None	None	None
52	Address	VARCHAR	YES	None	None	None
53	City	VARCHAR	YES	None	None	None
54	County	VARCHAR	YES	None	None	None
55	DepartmentName	VARCHAR	YES	None	None	None
56	DepartmentSpecialty	VARCHAR	YES	None	None	None
57	DepartmentType	VARCHAR	YES	None	None	None
58	PostalCode	VARCHAR	YES	None	None	None
59	CensusTract	VARCHAR	YES	None	None	None

```
In [34]: duckdb.sql("SELECT COUNT(*) FROM combined_data")
```

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

Out[34]:

count_star() int64
7675801

Checking Specific Diagnosis

```
In [37]: pd.set_option('display.max_rows', None)
```

```
In [39]: duckdb.sql("""
SELECT GroupName, COUNT(*) as Count
FROM combined_data
WHERE DepartmentType = 'ED'
GROUP BY GroupName
ORDER BY Count DESC
""").df().head(10)
```

FloatProgress(value=0.0, layout=Layout(width='auto'), style=ProgressStyle(bar_color='black'))

Out[39]:

	GroupName	Count
0	Abdominal and pelvic pain	11851
1	Pain in throat and chest	10916
2	None	8345
3	Persons encountering health services for specific procedures and treatment, not carried out	7360
4	Dorsalgia	6943
5	Other joint disorder, not elsewhere classified	6136
6	Nausea and vomiting	4465
7	Other and unspecified soft tissue disorders, not elsewhere classified	4214
8	Open wound of head	3502
9	COVID-19	2832

```
In [ ]:
```

What I plan on looking closer into

While the top amount of patient count is General examination -- we will be ignoring that data. We will look into Emergency Departments, ["Abdominal and pelvic pain", "Pain in throat and chest"]

Will be moving to Data_visualization -->

In []: